VGA Graphics Controller

User Manual

Hardware IP Core for Xilinx FPGAs

Larry D. Pyeatt

Version 1.0

February 21, 2026

Dual-Resolution Display Controller
with Double-Buffering and Palette Support

Contents

List of Tables

List of Figures

Chapter 1

Introduction

1.1 About This Manual

This manual provides complete documentation for the VGA Graphics Controller, a dual-resolution display controller designed for Xilinx Field-Programmable Gate Arrays (FPGAs). This hardware IP core enables you to generate standard VGA video signals for connecting to monitors and displays.

Whether you're building a retro gaming console, creating custom graphics applications, developing embedded systems with display capabilities, or learning about digital display systems, this manual will guide you through every aspect of the VGA Graphics Controller.

1.2 What is the VGA Graphics Controller?

The VGA Graphics Controller is a synthesizable VHDL IP core that generates industry-standard VGA timing signals and manages framebuffer memory. It provides a complete solution for adding graphics capabilities to your FPGA design.

1.2.1 Key Features

- **Dual Resolution Modes:**
  - 320×200 pixels with 2×2 pixel doubling (displayed as 640×400)
  - 640×400 pixels native resolution
- **Color Support:**
  - 8-bit indexed color (256 simultaneous colors)
  - 12-bit RGB palette (4,096 possible colors: R4G4B4)
  - Programmable color lookup table (palette)
- **Memory Architecture:**
  - Internal framebuffer using FPGA Block RAM
  - 256 KB total framebuffer memory
  - Dual-port architecture for simultaneous CPU and display access
  - Double-buffering support in 320×200 mode
- **Interface:**
  - AXI4-Lite slave interface for CPU access
  - Memory-mapped framebuffer and palette
  - Simple register-based control
- **Display Specifications:**
  - 640×400 @ 70 Hz output
  - 25.175 MHz pixel clock
  - Standard VGA timing (VESA compatible)
  - Separate H-SYNC and V-SYNC outputs
  - 12-bit digital RGB output (4 bits per channel)

1.2.2 Target Applications

This VGA controller is ideal for:

- Retro gaming systems and consoles
- Embedded systems with display requirements
- Educational projects for learning computer graphics
- Industrial control panels and HMI systems
- FPGA-based soft processors (MicroBlaze, RISC-V, etc.)
- Test equipment and diagnostic displays
- Digital art and creative coding projects

1.3 Who Should Read This Manual?

This manual is intended for engineers, developers, and hobbyists working with:

- FPGA development and digital design
- Embedded systems programming
- Computer graphics and display systems
- Retro computing and gaming
- Educational projects in computer engineering

1.4 Prerequisites

1.4.1 Required Knowledge

To effectively use this VGA controller, you should be familiar with:

- FPGA development flow (synthesis, implementation, bitstream generation)
- Xilinx Vivado design tools
- Basic VHDL (for understanding the design, optional for use)
- C programming (for writing software to control the display)
- AXI bus protocol basics (helpful but not required)
- Basic understanding of raster graphics and frame buffers

1.4.2 Required Hardware

Minimum hardware requirements:

- Xilinx FPGA development board (Artix-7 or higher recommended)
- VGA connector (most dev boards include this)
- VGA-capable monitor supporting 640×400 @ 70 Hz
- VGA cable
- USB cable for FPGA programming

1.4.3 Required Software

- Xilinx Vivado Design Suite (2018.3 or later)
- Text editor or IDE for C programming
- (Optional) Xilinx SDK or Vitis for embedded software development

1.5 Document Conventions

Throughout this manual, the following conventions are used:

- `Monospace text` indicates code, register names, or memory addresses
- **Bold text** highlights important concepts or warnings
- *Italic text* introduces new terminology
- Hexadecimal values are prefixed with `0x`
- Binary values are prefixed with `0b`

Chapter 2

Hardware Overview

2.1 System Architecture

The VGA Graphics Controller consists of several key hardware blocks that work together to generate video signals and manage graphics memory.

2.1.1 Block Diagram

The complete system architecture is shown in Figure 2.1.

2.1.2 Component Descriptions

VGA Timing Generator

The timing generator produces all necessary VGA synchronization signals:

- Horizontal sync (H-SYNC) pulse
- Vertical sync (V-SYNC) pulse
- Pixel clock enable signals
- Video blanking control
- Framebuffer address generation

The timing generator operates at 25.175 MHz (the VGA pixel clock) and generates timing for 640×400 @ 70 Hz display mode. It automatically handles pixel doubling when in 320×200 mode.

Framebuffer BRAM

The framebuffer is implemented using FPGA Block RAM and stores pixel data:

- Total size: 256 KB
- Organization: Byte-addressable (8 bits per pixel)
- AXI Interface: Supports byte, halfword, and word writes (1, 2, or 4 bytes per transaction)
- Dual-port architecture:
  - Port A: Connected to AXI interface (CPU read/write)
  - Port B: Connected to VGA timing generator (display read)
- Supports two 320×200 buffers (double-buffering) or one 640×400 buffer

Color Palette RAM

The palette provides color lookup functionality:

- 256 entries (matching 8-bit pixel values)
- 12-bit RGB color depth per entry (4 bits per channel)
- Dual-port architecture for CPU and display access
- Programmable through AXI interface

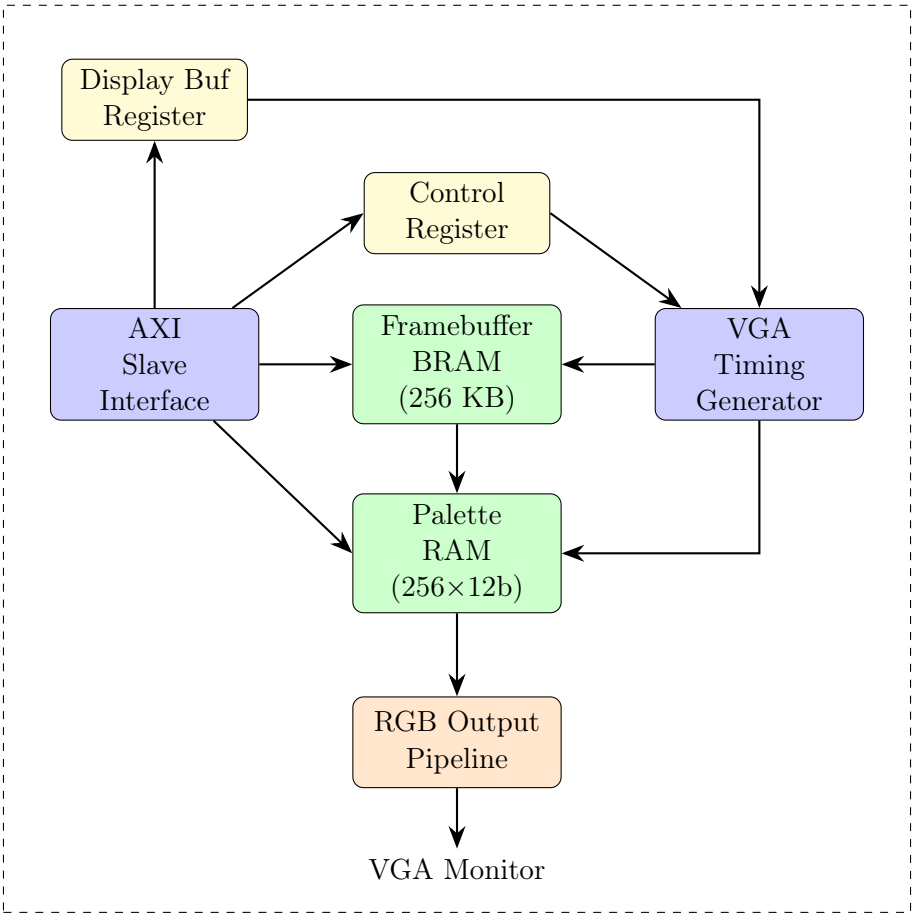


Figure 2.1: VGA Controller Block Diagram

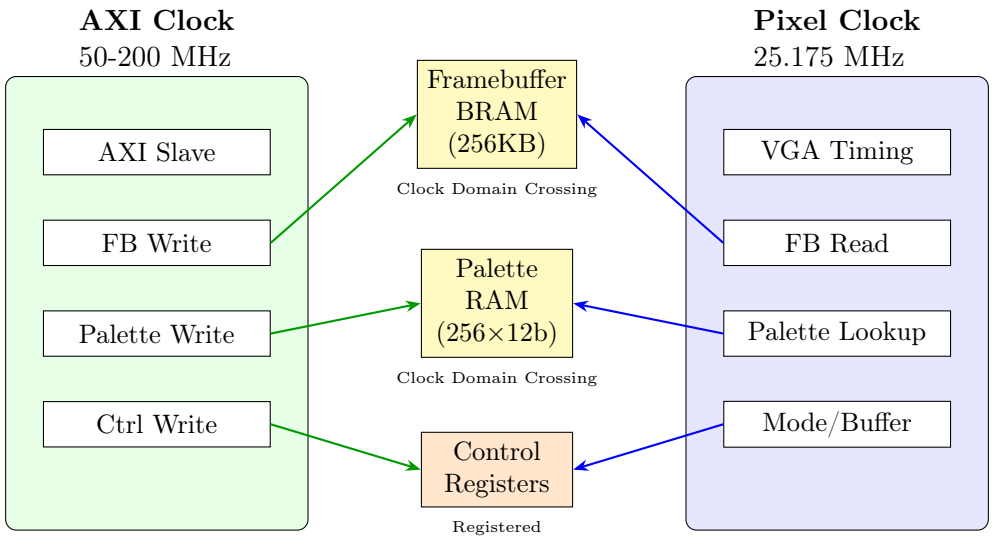


Figure 2.2: Clock Domain Architecture

AXI4-Lite Slave Interface

Provides CPU access to all controller resources:

- Memory-mapped framebuffer access
- Palette programming interface
- Control and status registers
- Standard AXI4-Lite protocol
- Low-latency access: 2 AXI cycles (registers/palette), 2-5 cycles (framebuffer multi-byte writes)

RGB Output Pipeline

Converts indexed pixel data to RGB signals:

- Two-stage pipeline for timing closure
- Palette lookup operation
- Video blanking insertion
- 12-bit RGB output (4 bits per channel)

2.2 Clock Domains

The VGA controller operates with two independent clock domains, as illustrated in Figure 2.2.

2.2.1 Pixel Clock Domain (25.175 MHz)

- VGA timing generation
- Framebuffer reads for display
- Palette lookups
- RGB output pipeline

This clock must be generated from your board's oscillator using a clock management primitive (MMCM or PLL). The provided TCL script automatically creates a clocking wizard configured for this frequency.

Table 2.1: FPGA Resource Utilization

Resource	Used	Available	Utilization
Total LUTs	1,487	63,400	2.35%
Logic LUTs	1,391	63,400	2.19%
LUTRAMs	96	19,000	0.51%
Flip-Flops	253	126,800	0.20%
Block RAM (RAMB36)	64	135	47.41%
DSP Blocks	0	240	0.00%
I/O Pins	14	210	6.67%

2.2.2 AXI Clock Domain (50-200 MHz)

- AXI4-Lite slave interface
- CPU framebuffer writes
- Palette updates
- Control register access

This clock typically comes from your processor or system interconnect. Clock domain crossing is handled automatically by the dual-port Block RAM.

2.3 Resource Utilization

Actual resource usage on Xilinx Artix-7 XC7A100T (post-synthesis):

VGA I/O Pins: 14 total (HSYNC, VSYNC, 4-bit Red, 4-bit Green, 4-bit Blue)

Note: Block RAM usage is the primary resource constraint, utilizing approximately 47% of available BRAM. Logic utilization remains low (<3%), leaving ample resources for additional features.

Chapter 3

Getting Started

3.1 Quick Start Guide

This section provides step-by-step instructions to get your VGA controller running quickly.

3.1.1 Hardware Setup

1. Connect your FPGA development board to your computer via USB
2. Connect a VGA monitor to the VGA port on your FPGA board
3. Power on the monitor
4. Ensure the monitor supports 640×400 @ 70 Hz (most modern monitors do)

3.1.2 Creating the Vivado Project

Using the Provided TCL Script (Recommended)

The easiest way to create a project is using the included TCL script:

1. Extract the VGA controller archive
2. Open Vivado
3. In the TCL console, navigate to the project directory:

```
cd /path/to/vga_controller
```

4. Run the project creation script:

```
/opt/Xilinx/2025.1/Vivado/bin/vivado -mode tcl -source create_project.
   tcl
```

5. The script will:
   - Create a new Vivado project
   - Add all VHDL source files
   - Configure the target FPGA (Nexys A7-100T by default)
   - Add pin constraints
   - Create a clocking wizard IP for the 25.175 MHz pixel clock

Manual Project Creation

If you prefer to create the project manually:

1. Create a new Vivado project
2. Select your target FPGA
3. Add all `.vhdl` files from the `vga_controller` directory
4. Set `vga_controller_top.vhdl` as the top module
5. Add the `.xdc` constraints file
6. Create a Clocking Wizard IP:
   - Input clock: Your board's frequency (e.g., 100 MHz)
   - Output clock: 25.175 MHz

Table 3.1: VGA Controller Pin Assignments

Signal	Direction	Description
`pixel_clk`	Input	25.175 MHz pixel clock
`axi_clk`	Input	AXI bus clock
`resetn`	Input	Active-low reset
`vga_hsync`	Output	Horizontal sync
`vga_vsync`	Output	Vertical sync
`vga_red[3:0]`	Output	Red channel (4 bits)
`vga_green[3:0]`	Output	Green channel (4 bits)
`vga_blue[3:0]`	Output	Blue channel (4 bits)

- Enable reset port (active low)

3.1.3 Pin Assignments

For the Digilent Nexys A7-100T board, pins are pre-configured in the XDC file. For other boards, you'll need to assign pins according to Table 3.1.

3.1.4 Building the Design

1. Click "Run Synthesis" in Vivado
2. Wait for synthesis to complete (typically 2–5 minutes)
3. Click "Run Implementation"
4. Wait for implementation to complete
5. Click "Generate Bitstream"
6. Wait for bitstream generation to complete

3.1.5 Programming the FPGA

1. Click "Open Hardware Manager"
2. Click "Auto Connect" to connect to your FPGA board
3. Right-click on the device and select "Program Device"
4. Select the generated `.bit` file
5. Click "Program"

3.1.6 First Test

At this point, the VGA controller is loaded but the framebuffer contains random data. To see meaningful output, you'll need to write software to control the display. See Chapter 6 for programming examples.

For a quick test without software:

- The display should show output (though it will be noise/random pixels)
- The monitor should recognize the 640×400 signal
- H-SYNC and V-SYNC should be working (monitor stays in sync)

If the monitor displays "No Signal" or "Out of Range," see Chapter 11.

Chapter 4

Memory Map Reference

4.1 Address Space Overview

The VGA controller is accessed through a single contiguous address space via the AXI4-Lite interface. The complete memory map is shown in Figure 4.1 and Table 4.1.

Table 4.1: Complete Memory Map

Address Range	Size	Description
0x00000-0x3FFFF	256 KB	Framebuffer Memory
0x40000-0x401FF	512 bytes	Color Palette (256 entries)
0x40200-0x40203	4 bytes	Control Register
0x40204-0x40207	4 bytes	Display Buffer Select Register
0x40208-0x4020B	4 bytes	IRQ Enable Register
0x4020C-0x4020F	4 bytes	IRQ Clear/Status Register

4.2 Framebuffer Memory (0x00000-0x3FFFF)

The framebuffer stores pixel data as 8-bit indexed color values. Each byte represents one pixel and is used as an index into the color palette.

4.2.1 Memory Layout

Pixels are stored in row-major order:

$$\text{Address} = y \times \text{width} + x \tag{4.1}$$

Where:
- x is the horizontal pixel coordinate (0 to width-1)
- y is the vertical pixel coordinate (0 to height-1)
- width is 320 (low-res mode) or 640 (high-res mode)

4.2.2 320×200 Mode (Double-Buffered)

In 320×200 mode, the framebuffer is divided into two buffers for double-buffering support as shown in Figure 4.2:

Address Calculation Example:

To write a pixel at coordinates (x=100, y=50) to Buffer 1:

$$\begin{aligned} \text{Address} &= \text{Buffer}_1\_\text{Base} + (y \times 320 + x) \\ &= \texttt{0x20000} + (50 \times 320 + 100) \\ &= \texttt{0x20000} + 16100 \\ &= \texttt{0x23EE4} \end{aligned} \tag{4.2}$$

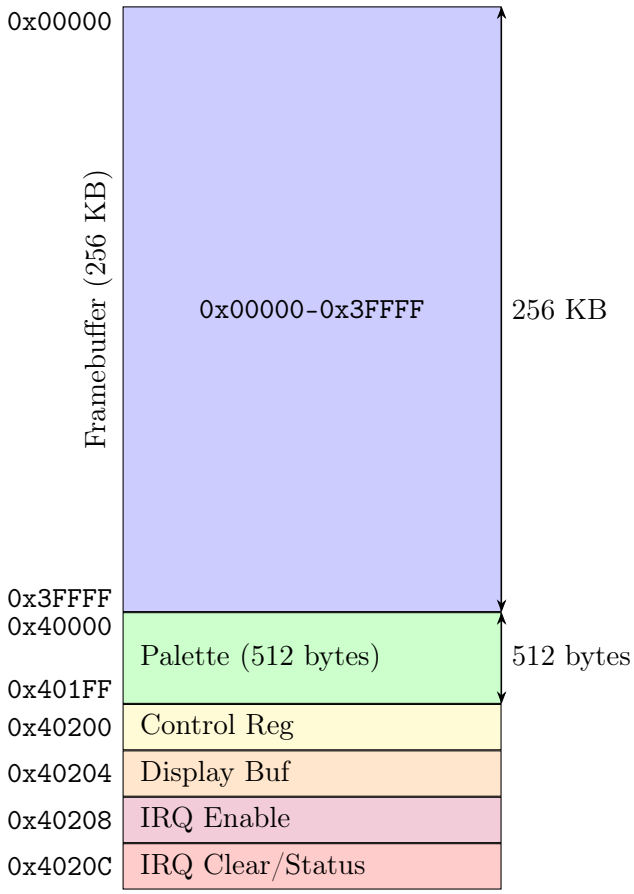


Figure 4.1: Memory Map Visualization

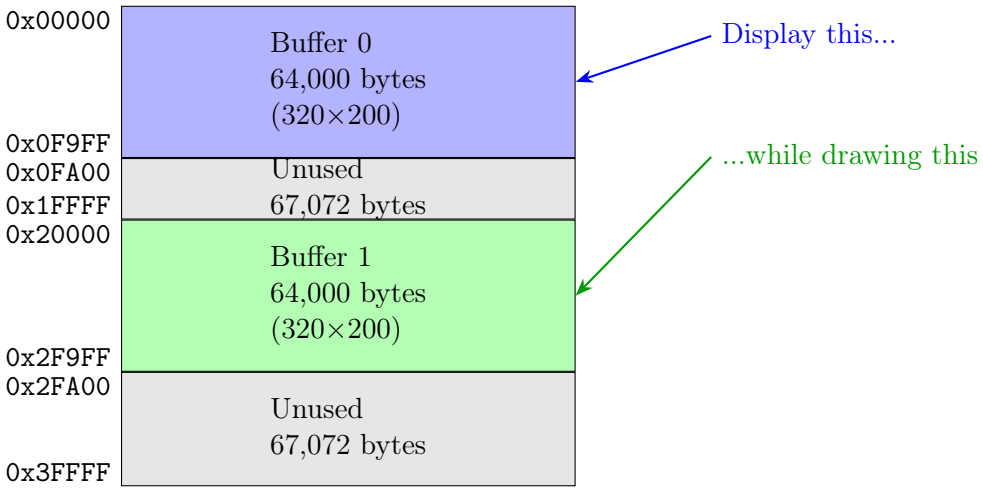


Figure 4.2: 320×200 Double-Buffer Layout

Table 4.2: 320×200 Framebuffer Layout

Buffer	Address Range	Size
Buffer 0	`0x00000-0x0F9FF`	64,000 bytes
Unused	`0x0FA00-0x1FFFF`	67,072 bytes
Buffer 1	`0x20000-0x2F9FF`	64,000 bytes
Unused	`0x2FA00-0x3FFFF`	67,072 bytes

Table 4.3: 640×400 Framebuffer Layout

Region	Address Range	Size
Framebuffer	`0x00000-0x3E7FF`	256,000 bytes
Unused	`0x3E800-0x3FFFF`	6,144 bytes

4.2.3 640×400 Mode (Single Buffer)

In 640×400 mode, a single framebuffer occupies the lower portion of memory:

Address Calculation Example:

To write a pixel at coordinates (x=320, y=200):

$$\begin{aligned} \text{Address} &= y \times 640 + x \\ &= 200 \times 640 + 320 \\ &= 128320 \\ &= \texttt{0x1F520} \end{aligned} \tag{4.3}$$

4.2.4 Access Characteristics

- **Write Access:** Supports byte, halfword, and word writes (1-4 bytes written sequentially)
- **Read Access:** Supports byte, halfword, and word reads (1-4 bytes read sequentially)
- **Width:** 8-bit pixels (one byte per pixel)
- **Alignment:** Byte-addressable, no alignment restrictions
- **Write Latency:** 2 AXI cycles (single byte), 3-5 cycles (multi-byte sequential)
- **Read Latency:** 2 AXI cycles (single byte), 3-5 cycles (multi-byte sequential)
- **Byte Enables:** Supported for partial writes

Multi-byte Read/Write: The framebuffer supports efficient multi-byte reads and writes. When reading or writing words (4 bytes) or halfwords (2 bytes), the AXI slave sequentially accesses consecutive framebuffer addresses while holding the bus busy. This provides optimal performance for bulk operations.

4.3 Color Palette (`0x40000-0x401FF`)

The color palette translates 8-bit pixel indices to 12-bit RGB colors.

4.3.1 Palette Entry Format

Each palette entry is 16 bits wide (16-bit halfword aligned in memory) as detailed in Table 4.4:

Table 4.4: Palette Entry Bit Fields

Bits	Field	Description
15–12	Reserved	Must be written as 0
11–8	Red	Red channel intensity (0–15)
7–4	Green	Green channel intensity (0–15)
3–0	Blue	Blue channel intensity (0–15)

Table 4.5: Standard 16-Color VGA Palette

Index	Color Name	RGB Value
0	Black	`0x000`
1	Blue	`0x00A`
2	Green	`0x0A0`
3	Cyan	`0x0AA`
4	Red	`0xA00`
5	Magenta	`0xA0A`
6	Brown	`0xA50`
7	Light Gray	`0xAAA`
8	Dark Gray	`0x555`
9	Bright Blue	`0x00F`
10	Bright Green	`0x0F0`
11	Bright Cyan	`0x0FF`
12	Bright Red	`0xF00`
13	Bright Magenta	`0xF0F`
14	Yellow	`0xFF0`
15	White	`0xFFF`

4.3.2 Palette Addressing

Palette entries are accessed at word-aligned addresses:

$$\text{Palette Address} = \texttt{0x40000} + (\text{color\_index} \times 2) \tag{4.4}$$

Example: To set color index 5 to bright red (RGB = F00):

```
uint16_t *palette = (uint16_t*)0x48040000;  // Base + palette offset
palette[5] = 0x0F00;  // Bright red
```

4.3.3 Default Palette

On reset, all palette entries are initialized to black (0x000). You must program the palette before meaningful colors appear.

4.3.4 Standard VGA Colors

A common starting palette uses the 16 standard VGA colors shown in Table 4.5.

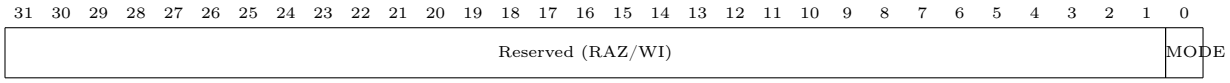
Chapter 5

Register Descriptions

5.1 Control Register (0x40200)

The Control Register configures the basic operating mode of the VGA controller. Table 5.1 shows the bit field definitions.

5.1.1 Register Fields



5.1.2 Reset Value

0x00000000 (640×400 mode)

5.1.3 Usage Example

```c
#define CTRL_REG  0x48040200

// Switch to 320x200 mode
*(volatile uint32_t*)CTRL_REG = 0x00000001;

// Switch to 640x400 mode
*(volatile uint32_t*)CTRL_REG = 0x00000000;

// Read current mode
uint32_t mode = *(volatile uint32_t*)CTRL_REG;
if (mode & 0x01) {
    printf("320x200 mode\n");
} else {
    printf("640x400 mode\n");
}
```

Listing 5.1: Setting Display Mode

Table 5.1: Control Register Bit Fields

Bit	Name	Type	Description
0	MODE	R/W	Resolution mode select 0 = 640×400 mode 1 = 320×200 mode
31–1	–	RAZ/WI	Reserved (Read-As-Zero, Write-Ignored)

Table 5.2: Display Buffer Register Bit Fields

Bit	Name	Type	Description
0	BUF	R/W	Display buffer select 0 = Display Buffer 0 1 = Display Buffer 1 (320×200 mode only)
31–1	–	RAZ/WI	Reserved

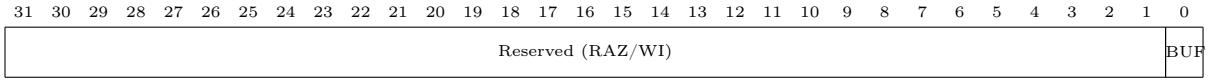
5.1.4 Notes

- Mode changes take effect immediately but may cause visual glitches
- It's recommended to change modes during vertical blanking
- When switching from 320×200 to 640×400, the active buffer setting is ignored
- Framebuffer contents are not cleared when changing modes

5.2 Display Buffer Register (0x40204)

The Display Buffer Register selects which framebuffer is displayed in 320×200 mode. This enables double-buffering for flicker-free animation as shown in Table 5.2.

5.2.1 Register Fields



5.2.2 Reset Value

0x00000000 (Buffer 0 displayed)

5.2.3 Usage Example

```
#define DISP_BUF_REG  0x48040204

uint8_t current_display_buffer = 0;
uint8_t current_draw_buffer = 1;

void swap_buffers(void) {
    // Swap the buffer being displayed
    current_display_buffer = current_draw_buffer;
    current_draw_buffer = 1 - current_draw_buffer;

    // Update hardware register (instant swap)
    *(volatile uint32_t*)DISP_BUF_REG = current_display_buffer;
}

// In your main animation loop:
while (1) {
    // Draw to the off-screen buffer
```

Table 5.3: IRQ Enable Register Bit Fields

Bit	Name	Type	Description
0	VSIEN	R/W	Vertical Sync Interrupt Enable 0 = Interrupt output disabled 1 = Interrupt output enabled
31–1	–	RAZ/WI	Reserved

```
18      draw_frame(current_draw_buffer);
19
20      // Swap buffers (no tearing!)
21      swap_buffers();
22
23      // Optional: wait for vsync
24      wait_for_vsync();
25  }
```

Listing 5.2: Double-Buffering Example

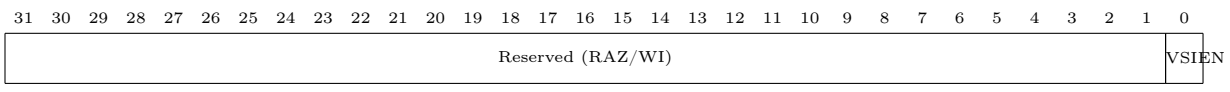
5.2.4 Notes

- **Immediate switching:** Buffer switching is instantaneous - the change takes effect on the very next pixel being displayed
- **Warning:** Switching mid-frame will cause visible tearing (a horizontal line where the buffer change occurred)
- **Recommended practice:** Always swap buffers during vertical blanking using the VSYNC interrupt for tear-free animation
- This register has no effect in 640×400 mode
- Reading this register returns the currently displayed buffer

5.3 IRQ Enable Register (0x40208)

The IRQ Enable Register controls whether the VSYNC interrupt is forwarded to the processor. The interrupt fires at the beginning of the vertical blanking interval, making it ideal for synchronizing double-buffer swaps and other timing-critical operations.

5.3.1 Register Fields



5.3.2 Reset Value

0x00000000 (Interrupt disabled)

5.3.3 Notes

- VSIEN gates the `irq` output: `irq` = VSIEN AND VSIA

Table 5.4: IRQ Clear/Status Register Bit Fields

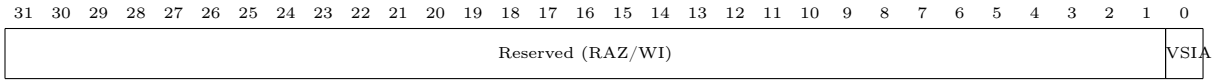
Bit	Name	Type	Description
0	VSIA	R/W1C	Vertical Sync Interrupt Active 0 = No interrupt pending 1 = Interrupt pending Write 1 to clear; writing 0 has no effect
31–1	–	RAZ/WI	Reserved

- Clearing VSIEN does *not* clear the pending flag in the IRQ Clear/Status Register
- When VSIEN is re-enabled, any pending event that occurred while disabled will immediately assert `irq`
- Write 0 to this register to disable the interrupt at any time

5.4 IRQ Clear/Status Register (0x4020C)

The IRQ Clear/Status Register reflects the current VSYNC interrupt pending state and provides the write-1-to-clear mechanism for acknowledging the interrupt. Reading this register allows software to poll for VSYNC without using the interrupt output. Table 5.4 details the bit field definitions.

5.4.1 Register Fields



5.4.2 Operation

Interrupt Generation:

- VSIA is automatically set on the rising edge of VSYNC (start of vertical blanking)
- The `irq` output = VSIEN (from IRQ Enable Register) AND VSIA
- The interrupt remains asserted until software clears VSIA
- VSIA is set regardless of whether VSIEN is enabled

Clearing the Interrupt:

- Write `0x00000001` to this register to clear VSIA
- Writing 0 to bit 0 has no effect (write-1-to-clear semantics)
- The NVIC pending bit must also be cleared in the interrupt handler

5.4.3 Reset Value

`0x00000000` (No interrupt pending)

5.4.4 Usage Example

```
#define VGA_BASE             0x48000000
#define VGA_IRQ_ENABLE_REG   (VGA_BASE + 0x40208)  // VSIEN at bit 0
#define VGA_IRQ_CLEAR_REG    (VGA_BASE + 0x4020C)  // VSIA  at bit 0 (R/W1C
    )
```

```c
#define VGA_DISP_BUF_REG    (VGA_BASE + 0x40204)

// Enable VGA VSYNC interrupt
void vga_enable_irq(void) {
    *(volatile uint32_t*)VGA_IRQ_ENABLE_REG = 0x01;  // Set VSIEN
}

// VGA interrupt handler (called on VSYNC rising edge)
void vga_irq_handler(void) {
    // Check VSIA in the Clear/Status register
    if (*(volatile uint32_t*)VGA_IRQ_CLEAR_REG & 0x01) {
        // VSYNC occurred -- safe to swap buffers now
        static uint8_t display_buffer = 0;
        display_buffer ^= 1;
        *(volatile uint32_t*)VGA_DISP_BUF_REG = display_buffer;

        // Acknowledge: write 1 to bit 0 of IRQ Clear/Status register
        *(volatile uint32_t*)VGA_IRQ_CLEAR_REG = 0x01;
    }
}

// Main application
int main(void) {
    vga_enable_irq();
    register_interrupt_handler(VGA_IRQ_NUM, vga_irq_handler);
    enable_global_interrupts();

    while (1) {
        // Draw to off-screen buffer
        // Buffer swap happens automatically in the IRQ handler
        draw_next_frame();
    }
}
```

Listing 5.3: Interrupt-Driven Double-Buffering

5.4.5 Typical Use Case: Tear-Free Graphics

The interrupt-driven approach ensures buffer swaps occur exclusively during vertical blanking:

1. Write 1 to VSIEN in the IRQ Enable Register
2. Draw graphics to the off-screen buffer
3. On VSYNC interrupt:
   - Swap the Display Buffer Register (no tearing — the scan-out is in blanking)
   - Optionally trigger audio or timing updates
   - Write `0x01` to the IRQ Clear/Status Register to clear VSIA
4. Repeat

5.4.6 Notes

- The interrupt fires at approximately 70 Hz (once per frame)
- VSYNC occurs during the vertical blanking interval when no pixels are being drawn
- This is the optimal time to swap buffers for tear-free animation

- The interrupt handler should be kept short to avoid missing the blanking window
- Reading bit 0 of this register returns the current pending status
- The `irq` output connects to your processor's interrupt controller

5.5 Status Registers (Future)

The current version does not include status registers, but future versions may add:

- V-SYNC status bit (for software synchronization)
- Frame counter
- Performance counters

Chapter 6

Programming Guide

6.1 Software Architecture

This chapter provides complete programming information for the VGA controller, including initialization, drawing primitives, and advanced techniques.

6.2 Initialization

6.2.1 Memory-Mapped Base Address

First, define the base address where the VGA controller is mapped in your system:

```
// Adjust this address to match your system configuration
#define VGA_BASE            0x48000000

// Derived addresses
#define FB_BASE             (VGA_BASE + 0x00000)
#define FB_BUFFER_0         (FB_BASE  + 0x00000)  // 320x200 buffer 0
#define FB_BUFFER_1         (FB_BASE  + 0x20000)  // 320x200 buffer 1 (128
    KB boundary)
#define PALETTE_BASE        (VGA_BASE + 0x40000)
#define CTRL_REG            (VGA_BASE + 0x40200)
#define DISP_BUF_REG        (VGA_BASE + 0x40204)
#define VGA_IRQ_ENABLE_REG  (VGA_BASE + 0x40208)  // VSIEN at bit 0
#define VGA_IRQ_CLEAR_REG   (VGA_BASE + 0x4020C)  // VSIA  at bit 0 (R/W1C
   )
```

6.2.2 Basic Initialization Sequence

```
void vga_init(void) {
    // 1. Set display mode
    *(volatile uint32_t*)CTRL_REG = 0x00000001;  // 320x200 mode

    // 2. Initialize palette with default colors
    init_palette();

    // 3. Clear both buffers
    clear_buffer(0);
    clear_buffer(1);

    // 4. Select buffer 0 for display
    *(volatile uint32_t*)DISP_BUF_REG = 0x00000000;
}

void clear_buffer(uint8_t buffer) {
    volatile uint8_t *fb;
```

```
18
19      if (buffer == 0) {
20          fb = (volatile uint8_t*)FB_BUFFER_0;
21      } else {
22          fb = (volatile uint8_t*)FB_BUFFER_1;
23      }
24
25      // Clear 64,000 bytes (320x200)
26      for (int i = 0; i < 64000; i++) {
27          fb[i] = 0;  // Black (assuming palette[0] is black)
28      }
29  }
```

Listing 6.1: VGA Controller Initialization

6.2.3 Palette Initialization

```
1   void init_palette(void) {
2       volatile uint32_t *palette = (volatile uint32_t*)PALETTE_BASE;
3
4       // Standard 16-color VGA palette
5       palette[0]  = 0x000;  // Black
6       palette[1]  = 0x00A;  // Blue
7       palette[2]  = 0x0A0;  // Green
8       palette[3]  = 0x0AA;  // Cyan
9       palette[4]  = 0xA00;  // Red
10      palette[5]  = 0xA0A;  // Magenta
11      palette[6]  = 0xA50;  // Brown
12      palette[7]  = 0xAAA;  // Light Gray
13      palette[8]  = 0x555;  // Dark Gray
14      palette[9]  = 0x00F;  // Bright Blue
15      palette[10] = 0x0F0;  // Bright Green
16      palette[11] = 0x0FF;  // Bright Cyan
17      palette[12] = 0xF00;  // Bright Red
18      palette[13] = 0xF0F;  // Bright Magenta
19      palette[14] = 0xFF0;  // Yellow
20      palette[15] = 0xFFF;  // White
21
22      // Fill remaining entries with grayscale
23      for (int i = 16; i < 256; i++) {
24          uint8_t gray = i;  // 0-255 range
25          uint8_t level = gray >> 4;  // Convert to 0-15
26          palette[i] = (level << 8) | (level << 4) | level;
27      }
28  }
```

Listing 6.2: Standard VGA Palette Initialization

6.3 Drawing Primitives

6.3.1 Plot Pixel

```c
void plot_pixel_320(uint16_t x, uint16_t y,
                    uint8_t color, uint8_t buffer) {
    if (x >= 320 || y >= 200) return;  // Bounds check

    volatile uint8_t *fb;
    if (buffer == 0) {
        fb = (volatile uint8_t*)FB_BUFFER_0;
    } else {
        fb = (volatile uint8_t*)FB_BUFFER_1;
    }

    fb[y * 320 + x] = color;
}

void plot_pixel_640(uint16_t x, uint16_t y, uint8_t color) {
    if (x >= 640 || y >= 400) return;

    volatile uint8_t *fb = (volatile uint8_t*)FB_BASE;
    fb[y * 640 + x] = color;
}
```

Listing 6.3: Pixel Plotting Function

6.3.2 Read Pixel

```c
uint8_t get_pixel_320(uint16_t x, uint16_t y, uint8_t buffer) {
    if (x >= 320 || y >= 200) return 0;

    volatile uint8_t *fb;
    if (buffer == 0) {
        fb = (volatile uint8_t*)FB_BUFFER_0;
    } else {
        fb = (volatile uint8_t*)FB_BUFFER_1;
    }

    return fb[y * 320 + x];
}
```

Listing 6.4: Reading Pixel Values

6.3.3 Draw Line (Bresenham's Algorithm)

```c
void draw_line(int x0, int y0, int x1, int y1,
               uint8_t color, uint8_t buffer) {
    int dx = abs(x1 - x0);
    int dy = abs(y1 - y0);
    int sx = (x0 < x1) ? 1 : -1;
    int sy = (y0 < y1) ? 1 : -1;
    int err = dx - dy;

    while (1) {
```

```
        plot_pixel_320(x0, y0, color, buffer);

        if (x0 == x1 && y0 == y1) break;

        int e2 = 2 * err;
        if (e2 > -dy) {
            err -= dy;
            x0 += sx;
        }
        if (e2 < dx) {
            err += dx;
            y0 += sy;
        }
    }
}
```

Listing 6.5: Line Drawing

6.3.4 Draw Rectangle

```
void draw_rect(int x, int y, int w, int h,
               uint8_t color, uint8_t buffer) {
    // Top and bottom edges
    for (int i = 0; i < w; i++) {
        plot_pixel_320(x + i, y, color, buffer);
        plot_pixel_320(x + i, y + h - 1, color, buffer);
    }

    // Left and right edges
    for (int i = 0; i < h; i++) {
        plot_pixel_320(x, y + i, color, buffer);
        plot_pixel_320(x + w - 1, y + i, color, buffer);
    }
}

void fill_rect(int x, int y, int w, int h,
               uint8_t color, uint8_t buffer) {
    volatile uint8_t *fb;
    if (buffer == 0) {
        fb = (volatile uint8_t*)FB_BUFFER_0;
    } else {
        fb = (volatile uint8_t*)FB_BUFFER_1;
    }

    for (int row = y; row < y + h; row++) {
        if (row < 0 || row >= 200) continue;

        for (int col = x; col < x + w; col++) {
            if (col < 0 || col >= 320) continue;
            fb[row * 320 + col] = color;
        }
    }
}
```

Listing 6.6: Rectangle Drawing

6.3.5 Draw Circle (Midpoint Algorithm)

```
void plot_circle_points(int xc, int yc, int x, int y,
                        uint8_t color, uint8_t buffer) {
    plot_pixel_320(xc + x, yc + y, color, buffer);
    plot_pixel_320(xc - x, yc + y, color, buffer);
    plot_pixel_320(xc + x, yc - y, color, buffer);
    plot_pixel_320(xc - x, yc - y, color, buffer);
    plot_pixel_320(xc + y, yc + x, color, buffer);
    plot_pixel_320(xc - y, yc + x, color, buffer);
    plot_pixel_320(xc + y, yc - x, color, buffer);
    plot_pixel_320(xc - y, yc - x, color, buffer);
}

void draw_circle(int xc, int yc, int radius,
                 uint8_t color, uint8_t buffer) {
    int x = 0;
    int y = radius;
    int d = 1 - radius;

    plot_circle_points(xc, yc, x, y, color, buffer);

    while (x < y) {
        x++;
        if (d < 0) {
            d += 2 * x + 1;
        } else {
            y--;
            d += 2 * (x - y) + 1;
        }
        plot_circle_points(xc, yc, x, y, color, buffer);
    }
}
```

Listing 6.7: Circle Drawing

6.4 Advanced Techniques

6.4.1 Optimized Block Fill

For better performance when filling large areas:

```
#include <string.h>

void fast_fill_rect(int x, int y, int w, int h,
                    uint8_t color, uint8_t buffer) {
    volatile uint8_t *fb;
    if (buffer == 0) {
        fb = (volatile uint8_t*)FB_BUFFER_0;
```

```
    } else {
        fb = (volatile uint8_t*)FB_BUFFER_1;
    }

    for (int row = y; row < y + h; row++) {
        if (row < 0 || row >= 200) continue;

        int start_x = (x < 0) ? 0 : x;
        int end_x = (x + w > 320) ? 320 : x + w;
        int width = end_x - start_x;

        if (width > 0) {
            memset((void*)&fb[row * 320 + start_x], color, width);
        }
    }
}
```

Listing 6.8: Optimized Block Fill Using memset

6.4.2 Sprite Drawing

```
typedef struct {
    int width;
    int height;
    const uint8_t *data;
} Sprite;

void draw_sprite(int x, int y, const Sprite *sprite,
                 uint8_t buffer) {
    volatile uint8_t *fb;
    if (buffer == 0) {
        fb = (volatile uint8_t*)FB_BUFFER_0;
    } else {
        fb = (volatile uint8_t*)FB_BUFFER_1;
    }

    for (int row = 0; row < sprite->height; row++) {
        int screen_y = y + row;
        if (screen_y < 0 || screen_y >= 200) continue;

        for (int col = 0; col < sprite->width; col++) {
            int screen_x = x + col;
            if (screen_x < 0 || screen_x >= 320) continue;

            uint8_t pixel = sprite->data[row * sprite->width + col];

            // Color 0 is transparent
            if (pixel != 0) {
                fb[screen_y * 320 + screen_x] = pixel;
            }
        }
    }
}
```

```

// Example sprite data (8x8 smiley face)
const uint8_t smiley_data[64] = {
    0,0,15,15,15,15,0,0,
    0,15,14,14,14,14,15,0,
    15,14,1,14,14,1,14,15,
    15,14,14,14,14,14,14,15,
    15,14,1,14,14,1,14,15,
    15,14,14,1,1,14,14,15,
    0,15,14,14,14,14,15,0,
    0,0,15,15,15,15,0,0
};

const Sprite smiley = {
    .width = 8,
    .height = 8,
    .data = smiley_data
};
```

Listing 6.9: Sprite Rendering with Transparency

6.4.3 Scrolling

```
void scroll_up(int lines, uint8_t buffer) {
    volatile uint8_t *fb;
    if (buffer == 0) {
        fb = (volatile uint8_t*)FB_BUFFER_0;
    } else {
        fb = (volatile uint8_t*)FB_BUFFER_1;
    }

    // Move pixels up
    memmove((void*)fb,
            (const void*)&fb[lines * 320],
            (200 - lines) * 320);

    // Clear bottom lines
    memset((void*)&fb[(200 - lines) * 320], 0, lines * 320);
}
```

Listing 6.10: Vertical Scrolling

6.5 Double-Buffering Example

```
#include <stdint.h>
#include <stdbool.h>

typedef struct {
    uint8_t display_buffer;
    uint8_t draw_buffer;
    bool vsync_enabled;
```

```
} VGA_State;

VGA_State vga_state = {0, 1, false};

void vga_swap_buffers(void) {
    // Swap display and draw buffers
    uint8_t temp = vga_state.display_buffer;
    vga_state.display_buffer = vga_state.draw_buffer;
    vga_state.draw_buffer = temp;

    // Update hardware register
    *(volatile uint32_t*)DISP_BUF_REG = vga_state.display_buffer;
}

uint8_t vga_get_draw_buffer(void) {
    return vga_state.draw_buffer;
}

// Example: Bouncing ball animation
typedef struct {
    float x, y;
    float vx, vy;
    uint8_t color;
    int radius;
} Ball;

void animate_ball(Ball *ball) {
    const int WIDTH = 320;
    const int HEIGHT = 200;

    while (1) {
        uint8_t buffer = vga_get_draw_buffer();

        // Clear screen
        fast_fill_rect(0, 0, WIDTH, HEIGHT, 0, buffer);

        // Update ball position
        ball->x += ball->vx;
        ball->y += ball->vy;

        // Bounce off walls
        if (ball->x - ball->radius < 0 ||
            ball->x + ball->radius >= WIDTH) {
            ball->vx = -ball->vx;
        }
        if (ball->y - ball->radius < 0 ||
            ball->y + ball->radius >= HEIGHT) {
            ball->vy = -ball->vy;
        }

        // Draw ball
        draw_circle((int)ball->x, (int)ball->y,
                    ball->radius, ball->color, buffer);

```

```
        // Swap buffers (no tearing!)
        vga_swap_buffers();

        // Optional: delay for frame rate control
        delay_ms(16);  // ~60 FPS
    }
}

int main(void) {
    // Initialize VGA
    vga_init();

    // Create a ball
    Ball ball = {
        .x = 160,
        .y = 100,
        .vx = 2.5,
        .vy = 1.8,
        .color = 12,  // Bright red
        .radius = 8
    };

    // Run animation
    animate_ball(&ball);

    return 0;
}
```

Listing 6.11: Complete Double-Buffering Application

Chapter 7

Display Modes

7.1 320×200 Mode

7.1.1 Overview

320×200 mode provides a classic low-resolution mode suitable for retro gaming and applications where large pixels are desirable.

7.1.2 Technical Details

- **Resolution:** 320×200 pixels
- **Display:** Each pixel is doubled to 2×2 screen pixels (640×400 output)
- **Framebuffer Size:** 64,000 bytes per buffer
- **Double-Buffering:** Supported (two 64 KB buffers)
- **Refresh Rate:** 70 Hz
- **Colors:** 256 simultaneous from 4096-color palette

7.1.3 Memory Usage

7.1.4 Performance Characteristics

- **Pixel Update Rate:** Limited by AXI bus speed (typically 100 MHz)
- **Full Screen Update:** Can update entire framebuffer in <1 ms
- **Buffer Swap:** Instantaneous (single register write)
- **Memory Bandwidth:** 4.48 MB/s for display reads

7.1.5 Best Use Cases

- Retro-style games
- Pixel art applications
- Fast-paced animations requiring double-buffering
- Applications where memory is constrained
- Classic computer emulation

Table 7.1: 320×200 Mode Memory Allocation

Resource	Size	Usage
Buffer 0	64,000 bytes	25% of 256 KB
Buffer 1	64,000 bytes	25% of 256 KB
Total Used	128,000 bytes	50% of 256 KB
Available	128,000 bytes	50% of 256 KB

Table 7.2: 640×400 Mode Memory Allocation

Resource	Size	Usage
Framebuffer	256,000 bytes	100% of 256 KB
Available	0 bytes	0% of 256 KB

7.2 640×400 Mode

7.2.1 Overview

640×400 mode provides higher resolution for applications requiring more detail.

7.2.2 Technical Details

- **Resolution:** 640×400 pixels
- **Display:** 1:1 pixel mapping (no doubling)
- **Framebuffer Size:** 256,000 bytes
- **Double-Buffering:** Not supported (insufficient memory)
- **Refresh Rate:** 70 Hz
- **Colors:** 256 simultaneous from 4096-color palette

7.2.3 Memory Usage

7.2.4 Performance Characteristics

- **Full Screen Update:** 2.5 ms at 100 MHz AXI clock
- **Memory Bandwidth:** 17.92 MB/s for display reads
- **Pixel Density:** 4× more pixels than 320×200 mode

7.2.5 Best Use Cases

- Text-based interfaces requiring higher resolution
- Detailed graphics applications
- CAD/drawing applications
- Data visualization
- Applications where double-buffering is not needed

7.2.6 Handling Tearing Without Double-Buffering

Since 640×400 mode doesn't support double-buffering, you can minimize tearing by:

1. Drawing only during vertical blanking (if you can draw fast enough)
2. Using partial screen updates (only redraw changed regions)
3. Implementing triple-buffering in external SDRAM
4. Accepting some tearing for non-critical applications

7.3 Mode Comparison

Table 7.3 provides a detailed comparison of the two available display modes:

Table 7.3: Display Mode Comparison

Feature	**320×200**	**640×400**
Pixels	64,000	256,000
Memory per Buffer	64 KB	256 KB
Double-Buffering	Yes	No
Pixel Size	2×2 (large)	1×1 (small)
Update Speed	Fast	Moderate
Memory Available	50% free	0% free
Best For	Games	Applications

Chapter 8

Color Palette

8.1 Palette Architecture

The VGA controller uses an indexed color system with a programmable palette.

8.1.1 How It Works

The indexed color pipeline is illustrated in Figure 8.1:

1. Each pixel in the framebuffer is 8 bits (0–255)
2. This value is used as an index into the palette
3. The palette entry contains a 12-bit RGB color
4. The RGB color is output to the VGA DAC

8.1.2 Advantages of Indexed Color

- **Memory Efficiency:** 1 byte per pixel vs 3 bytes for direct RGB
- **Fast Color Changes:** Change palette to recolor entire screen
- **Palette Animation:** Create effects by cycling palette entries
- **Color Standardization:** Use consistent colors across application

8.2 Palette Programming

8.2.1 Setting Individual Colors

```
void set_palette_entry(uint8_t index,
                       uint8_t red, uint8_t green, uint8_t blue) {
    volatile uint32_t *palette = (volatile uint32_t*)PALETTE_BASE;

    // Combine RGB into 12-bit value (4 bits each)
    uint16_t rgb = ((red & 0x0F) << 8) |
                   ((green & 0x0F) << 4) |
                   (blue & 0x0F);

    palette[index] = rgb;
```

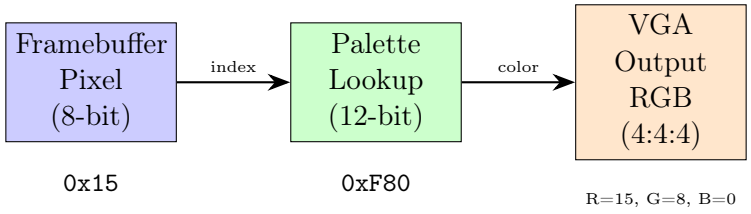


Figure 8.1: Indexed Color Pipeline

```
}

// Example: Set color 10 to orange
set_palette_entry(10, 15, 8, 0);  // RGB = F80
```

Listing 8.1: Setting Palette Entries

8.2.2 Loading Complete Palettes

```
typedef struct {
    uint8_t r, g, b;
} RGB;

void load_palette(const RGB *colors, int num_colors) {
    volatile uint32_t *palette = (volatile uint32_t*)PALETTE_BASE;

    for (int i = 0; i < num_colors && i < 256; i++) {
        uint16_t rgb = ((colors[i].r & 0x0F) << 8) |
                       ((colors[i].g & 0x0F) << 4) |
                       (colors[i].b & 0x0F);
        palette[i] = rgb;
    }
}

// Example: Earth tone palette
const RGB earth_palette[] = {
    {0, 0, 0},      // 0: Black
    {5, 3, 1},      // 1: Dark brown
    {8, 5, 2},      // 2: Brown
    {11, 8, 4},     // 3: Light brown
    {2, 6, 2},      // 4: Dark green
    {4, 10, 4},     // 5: Green
    {8, 13, 8},     // 6: Light green
    {10, 10, 15},   // 7: Sky blue
    {15, 15, 15},   // 8: White
};

load_palette(earth_palette, 9);
```

Listing 8.2: Loading a Full Palette

8.3 Palette Techniques

8.3.1 Palette Fading

```
void fade_to_black(int steps, int delay_ms) {
    RGB original_palette[256];
    volatile uint32_t *palette = (volatile uint32_t*)PALETTE_BASE;

    // Save original palette
    for (int i = 0; i < 256; i++) {
```

```
        uint16_t rgb = palette[i];
        original_palette[i].r = (rgb >> 8) & 0x0F;
        original_palette[i].g = (rgb >> 4) & 0x0F;
        original_palette[i].b = rgb & 0x0F;
    }

    // Fade out
    for (int step = steps; step >= 0; step--) {
        for (int i = 0; i < 256; i++) {
            uint8_t r = (original_palette[i].r * step) / steps;
            uint8_t g = (original_palette[i].g * step) / steps;
            uint8_t b = (original_palette[i].b * step) / steps;

            set_palette_entry(i, r, g, b);
        }
        delay_ms(delay_ms);
    }
}

// Restore original colors
void fade_from_black(const RGB *original, int steps, int delay_ms) {
    for (int step = 0; step <= steps; step++) {
        for (int i = 0; i < 256; i++) {
            uint8_t r = (original[i].r * step) / steps;
            uint8_t g = (original[i].g * step) / steps;
            uint8_t b = (original[i].b * step) / steps;

            set_palette_entry(i, r, g, b);
        }
        delay_ms(delay_ms);
    }
}
```

Listing 8.3: Fade to Black Effect

8.3.2 Palette Rotation (Color Cycling)

```
void rotate_palette_range(uint8_t start, uint8_t end, int direction) {
    volatile uint32_t *palette = (volatile uint32_t*)PALETTE_BASE;

    if (direction > 0) {
        // Rotate forward
        uint16_t temp = palette[end];
        for (int i = end; i > start; i--) {
            palette[i] = palette[i-1];
        }
        palette[start] = temp;
    } else {
        // Rotate backward
        uint16_t temp = palette[start];
        for (int i = start; i < end; i++) {
            palette[i] = palette[i+1];
        }
```

```
        palette[end] = temp;
    }
}

// Example: Animated water using palette cycling
void animate_water(void) {
    // Set up blue gradient in palette entries 16-31
    for (int i = 0; i < 16; i++) {
        uint8_t blue_level = i;
        set_palette_entry(16 + i, 0, i/2, blue_level);
    }

    // Draw water area using colors 16-31
    fill_rect(0, 150, 320, 50, 20, get_draw_buffer());

    // Animate by rotating palette
    while (1) {
        rotate_palette_range(16, 31, 1);
        delay_ms(50);  // Animation speed
    }
}
```

Listing 8.4: Water Effect Using Palette Rotation

8.3.3 Grayscale Conversion

```
void palette_to_grayscale(void) {
    volatile uint32_t *palette = (volatile uint32_t*)PALETTE_BASE;

    for (int i = 0; i < 256; i++) {
        uint16_t rgb = palette[i];
        uint8_t r = (rgb >> 8) & 0x0F;
        uint8_t g = (rgb >> 4) & 0x0F;
        uint8_t b = rgb & 0x0F;

        // Convert to grayscale using luminance formula
        // Gray = 0.299*R + 0.587*G + 0.114*B
        uint8_t gray = (3*r + 6*g + 1*b) / 10;

        palette[i] = (gray << 8) | (gray << 4) | gray;
    }
}
```

Listing 8.5: Convert Palette to Grayscale

8.4 Common Palette Schemes

8.4.1 Web-Safe Colors (216 colors)

```
void generate_websafe_palette(void) {
    int index = 0;
```

```
3
4       // 6x6x6 color cube
5       for (int r = 0; r < 6; r++) {
6           for (int g = 0; g < 6; g++) {
7               for (int b = 0; b < 6; b++) {
8                   // Map 0-5 to 0-15 (4-bit)
9                   uint8_t red = (r * 15) / 5;
10                  uint8_t green = (g * 15) / 5;
11                  uint8_t blue = (b * 15) / 5;
12
13                  set_palette_entry(index++, red, green, blue);
14
15                  if (index >= 256) return;
16              }
17          }
18      }
19  }
```

Listing 8.6: Generate Web-Safe Palette

Chapter 9

Double-Buffering

9.1 Concept

Double-buffering eliminates screen tearing by maintaining two framebuffers:

1. **Display Buffer:** Currently visible on screen
2. **Draw Buffer:** Being updated by the CPU

When drawing is complete, the buffers are swapped instantly, making the newly drawn frame visible without tearing or flicker, as shown in Figure 9.1.

9.1.1 Visual Explanation

9.2 Implementation

9.2.1 Basic Double-Buffer Manager

```c
typedef struct {
    uint8_t front_buffer;  // Currently displayed
    uint8_t back_buffer;   // Currently being drawn
} DoubleBuffer;

DoubleBuffer db = {0, 1};

void db_swap(void) {
    // Swap the buffers
    uint8_t temp = db.front_buffer;
    db.front_buffer = db.back_buffer;
    db.back_buffer = temp;

    // Update hardware to display new front buffer
    *(volatile uint32_t*)DISP_BUF_REG = db.front_buffer;
}

uint8_t db_get_back_buffer(void) {
    return db.back_buffer;
}
```

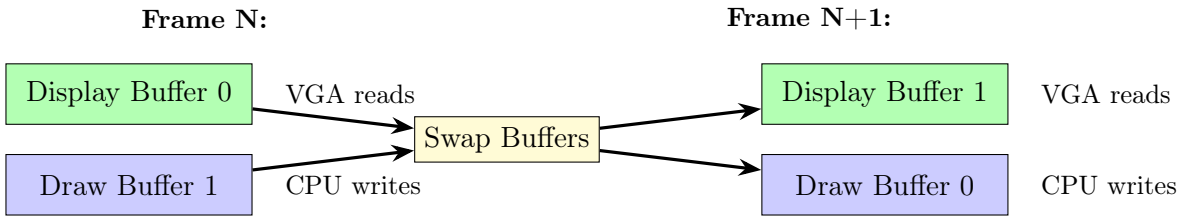


Figure 9.1: Double-Buffering Concept

```
21
22 uint8_t db_get_front_buffer(void) {
23     return db.front_buffer;
24 }
```

Listing 9.1: Double-Buffer Management

9.2.2 With V-Sync Synchronization

Important: The Display Buffer Register changes take effect *immediately* on the next pixel. Switching buffers mid-frame will cause visible screen tearing. To achieve tear-free animation, you *must* swap buffers during the vertical blanking interval using the VSYNC interrupt:

```
1  #define VGA_BASE             0x48000000
2  #define VGA_IRQ_ENABLE_REG   (VGA_BASE + 0x40208)  // VSIEN at bit 0
3  #define VGA_IRQ_CLEAR_REG    (VGA_BASE + 0x4020C)  // VSIA  at bit 0 (R/W1C
       )
4  #define DISP_BUF_REG         (VGA_BASE + 0x40204)
5
6  volatile uint8_t next_buffer = 0;
7  volatile uint8_t swap_pending = 0;
8
9  // Enable VSYNC interrupt
10 void init_vsync(void) {
11     *(volatile uint32_t*)VGA_IRQ_ENABLE_REG = 0x01;  // Set VSIEN
12 }
13
14 // VSYNC interrupt handler - swaps happen here (during vertical blanking)
15 void vga_irq_handler(void) {
16     // Check VSIA in the Clear/Status register
17     if (*(volatile uint32_t*)VGA_IRQ_CLEAR_REG & 0x01) {
18         // We're in vertical blanking - safe to swap now!
19         if (swap_pending) {
20             *(volatile uint32_t*)DISP_BUF_REG = next_buffer;
21             swap_pending = 0;
22         }
23
24         // Acknowledge interrupt (W1C: write 1 to bit 0)
25         *(volatile uint32_t*)VGA_IRQ_CLEAR_REG = 0x01;
26     }
27 }
28
29 // Request a buffer swap (will happen on next VSYNC)
30 void db_swap_request(void) {
31     next_buffer = 1 - next_buffer;  // Toggle buffer
32     swap_pending = 1;               // Mark swap as pending
33     // Actual swap happens in interrupt handler during vertical blanking
34 }
35
36 // Main animation loop
37 void animation_loop(void) {
38     init_vsync();
39
40     while (1) {
```

```
        // Draw to off-screen buffer
        draw_frame(1 - next_buffer);

        // Request buffer swap (happens on next VSYNC automatically)
        db_swap_request();

        // Continue - swap will happen in interrupt handler
    }
}
```

Listing 9.2: V-Sync Synchronized Buffer Swap Using Interrupt

Why This Works:

- The VSYNC interrupt fires during vertical blanking (when no pixels are being drawn)
- Swapping buffers during this time ensures the entire frame uses one consistent buffer
- No tearing occurs because the display never switches buffers mid-frame
- The interrupt-driven approach is more efficient than polling

What Happens Without VSYNC Synchronization:

- If you write to the Display Buffer Register at any time, the change is *immediate*
- The very next pixel displayed will come from the new buffer
- This creates a visible horizontal "tear line" where the buffer changed mid-frame
- Always use the interrupt handler approach shown above for tear-free graphics

9.2.3 Game Loop with Double-Buffering

```
typedef struct {
    float x, y;
    float dx, dy;
} GameObject;

void game_loop(void) {
    GameObject player = {160, 100, 0, 0};
    GameObject enemies[10];

    // Initialize
    vga_init();
    init_game_objects(enemies, 10);

    while (!game_over) {
        uint8_t buffer = db_get_back_buffer();

        // 1. Process input
        process_input(&player);

        // 2. Update game state
        update_player(&player);
        update_enemies(enemies, 10);
        check_collisions(&player, enemies, 10);

        // 3. Render to back buffer
        clear_screen(buffer);
        draw_player(&player, buffer);
        draw_enemies(enemies, 10, buffer);
```

```
        draw_ui(buffer);

        // 4. Swap buffers (makes new frame visible)
        db_swap_vsync();
    }
}
```

Listing 9.3: Typical Game Loop

9.3 Advanced Double-Buffering Techniques

9.3.1 Partial Updates

For static backgrounds, you can avoid redrawing the entire screen:

```
typedef struct {
    int x, y, w, h;
} DirtyRect;

#define MAX_DIRTY_RECTS 32

typedef struct {
    DirtyRect rects[MAX_DIRTY_RECTS];
    int count;
} DirtyRegions;

DirtyRegions dirty = {0};

void mark_dirty(int x, int y, int w, int h) {
    if (dirty.count < MAX_DIRTY_RECTS) {
        dirty.rects[dirty.count].x = x;
        dirty.rects[dirty.count].y = y;
        dirty.rects[dirty.count].w = w;
        dirty.rects[dirty.count].h = h;
        dirty.count++;
    }
}

void update_dirty_regions(uint8_t src_buffer, uint8_t dst_buffer) {
    volatile uint8_t *src_fb = (src_buffer == 0) ?
        (volatile uint8_t*)FB_BUFFER_0 :
        (volatile uint8_t*)FB_BUFFER_1;
    volatile uint8_t *dst_fb = (dst_buffer == 0) ?
        (volatile uint8_t*)FB_BUFFER_0 :
        (volatile uint8_t*)FB_BUFFER_1;

    for (int i = 0; i < dirty.count; i++) {
        DirtyRect *r = &dirty.rects[i];

        for (int y = r->y; y < r->y + r->h; y++) {
            memcpy((void*)&dst_fb[y * 320 + r->x],
                   (const void*)&src_fb[y * 320 + r->x],
                   r->w);
```

```
        }
    }

    dirty.count = 0;
}
```

Listing 9.4: Partial Screen Updates

9.3.2 Triple Buffering (Using External RAM)

If you have external SDRAM, you can implement triple buffering:

```
// Three buffers:
// - Buffer 0: Internal BRAM (currently displayed)
// - Buffer 1: Internal BRAM (ready to display)
// - Buffer 2: External SDRAM (being drawn)

typedef struct {
    uint8_t display_buffer;   // Currently on screen
    uint8_t ready_buffer;     // Ready to display
    uint8_t draw_buffer;      // Being drawn (in SDRAM)
} TripleBuffer;

TripleBuffer tb = {0, 1, 2};

void tb_swap(void) {
    // Move ready buffer to display
    tb.display_buffer = tb.ready_buffer;
    *(volatile uint32_t*)DISP_BUF_REG = tb.display_buffer;

    // Copy finished frame from SDRAM to ready buffer
    memcpy((void*)get_buffer_address(tb.ready_buffer),
           (void*)SDRAM_BUFFER_2,
           64000);

    // Rotate buffers
    uint8_t temp = tb.draw_buffer;
    tb.draw_buffer = tb.ready_buffer;
    tb.ready_buffer = temp;
}
```

Listing 9.5: Triple Buffering Concept

Chapter 10

Hardware Integration

10.1 Connecting to a MicroBlaze Processor

10.1.1 Block Design Creation

1. In Vivado, create a new Block Design
2. Add MicroBlaze processor IP
3. Run Block Automation (accept defaults)
4. Add VGA Controller:
   - Right-click in canvas, then Add Module
   - Select `vga_controller_top`
5. Add Clocking Wizard for 25.175 MHz pixel clock
6. Connect signals:
   - Connect MicroBlaze `M_AXI` to VGA `S_AXI`
   - Connect system clock to VGA `axi_clk`
   - Connect pixel clock to VGA `pixel_clk`
   - Connect reset to VGA `resetn`
   - Make VGA output signals external
7. Run Connection Automation
8. Assign Address (e.g., `0x48000000`, 1 MB range)
9. Validate Design
10. Create HDL Wrapper

10.1.2 Software Project Setup

```c
// vga.h
#ifndef VGA_H
#define VGA_H

#include <stdint.h>

// Base address from Vivado address editor
#define VGA_BASE            0x48000000

// Derived addresses
#define FB_BASE             (VGA_BASE + 0x00000)
#define FB_BUFFER_0         (FB_BASE  + 0x00000)  // 320x200 buffer 0
#define FB_BUFFER_1         (FB_BASE  + 0x20000)  // 320x200 buffer 1 (128
    KB boundary)
#define PALETTE_BASE        (VGA_BASE + 0x40000)
#define CTRL_REG            (VGA_BASE + 0x40200)
#define DISP_BUF_REG        (VGA_BASE + 0x40204)
#define VGA_IRQ_ENABLE_REG  (VGA_BASE + 0x40208)  // VSIEN at bit 0
#define VGA_IRQ_CLEAR_REG   (VGA_BASE + 0x4020C)  // VSIA  at bit 0 (R/W1C
   )
```

```

// Function prototypes
void vga_init(void);
void set_mode_320x200(void);
void set_mode_640x400(void);
void plot_pixel(int x, int y, uint8_t color, uint8_t buffer);
void set_palette_entry(uint8_t index, uint8_t r, uint8_t g, uint8_t b);

#endif // VGA_H
```

Listing 10.1: MicroBlaze Software Header

10.2 Standalone Configuration (No CPU)

For testing or simple applications, you can use the VGA controller without a processor:

10.2.1 Test Pattern Generator

```
-- test_pattern_gen.vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity test_pattern_gen is
    Port (
        clk         : in  std_logic;
        reset       : in  std_logic;
        fb_we       : out std_logic;
        fb_addr     : out std_logic_vector(18 downto 0);
        fb_din      : out std_logic_vector(7 downto 0);
        done        : out std_logic
    );
end test_pattern_gen;

architecture Behavioral of test_pattern_gen is
    signal addr_counter : unsigned(18 downto 0) := (others => '0');
    signal pattern_done : std_logic := '0';
begin
    process(clk)
    begin
        if rising_edge(clk) then
            if reset = '1' then
                addr_counter <= (others => '0');
                pattern_done <= '0';
            elsif pattern_done = '0' then
                -- Write test pattern (color bars)
                fb_we <= '1';
                fb_addr <= std_logic_vector(addr_counter);

                -- Generate horizontal color bars
                if addr_counter(9 downto 7) = "000" then
                    fb_din <= x"0F";  -- Blue
```

```
                elsif addr_counter(9 downto 7) = "001" then
                    fb_din <= x"0A";  -- Green
                elsif addr_counter(9 downto 7) = "010" then
                    fb_din <= x"0C";  -- Red
                else
                    fb_din <= x"0E";  -- Yellow
                end if;

                -- Increment address
                if addr_counter = 63999 then  -- 320x200
                    pattern_done <= '1';
                else
                    addr_counter <= addr_counter + 1;
                end if;
            else
                fb_we <= '0';
            end if;
        end if;
    end process;

    done <= pattern_done;
end Behavioral;
```

Listing 10.2: Simple Test Pattern Generator

10.3 Using with RISC-V Soft Core

The VGA controller can also be integrated with RISC-V processors like VexRiscv:

1. Add VexRiscv IP to your design
2. Configure VexRiscv with AXI4 master interface
3. Connect VGA controller to AXI interconnect
4. Compile RISC-V software using `riscv-gnu-toolchain`
5. Use the same C API as shown in this manual

10.4 Connecting to Zynq PS

For Zynq-7000 or Zynq UltraScale+ devices:

1. Add Zynq Processing System IP
2. Enable an AXI-GP port (e.g., M_AXI_GP0)
3. Add VGA controller to PL
4. Connect via AXI interconnect
5. Configure address map in PS
6. Access from Linux userspace using `/dev/mem` or UIO

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>

#define VGA_BASE_PHYS  0x48000000
```

```c
#define VGA_SIZE       0x00100000  // 1 MB

int main() {
    int fd = open("/dev/mem", O_RDWR | O_SYNC);
    if (fd < 0) {
        perror("open");
        return 1;
    }

    // Map VGA controller into process memory
    void *vga_base = mmap(NULL, VGA_SIZE,
                          PROT_READ | PROT_WRITE,
                          MAP_SHARED, fd, VGA_BASE_PHYS);

    if (vga_base == MAP_FAILED) {
        perror("mmap");
        close(fd);
        return 1;
    }

    // Now access as usual
    volatile uint8_t *fb = (volatile uint8_t*)vga_base;
    volatile uint32_t *ctrl = (volatile uint32_t*)(vga_base + 0x40200);

    // Set 320x200 mode
    *ctrl = 0x00000001;

    // Draw test pattern
    for (int i = 0; i < 64000; i++) {
        fb[i] = i % 256;
    }

    // Cleanup
    munmap(vga_base, VGA_SIZE);
    close(fd);

    return 0;
}
```

Listing 10.3: Linux Userspace Access

Chapter 11

Troubleshooting

11.1 No Display / Monitor Shows "No Signal"

11.1.1 Possible Causes and Solutions

1. **Pixel clock not running**
   - Verify clocking wizard is properly configured
   - Check that `locked` output of clock wizard is high
   - Verify 25.175 MHz output frequency
   - Check clock routing in design
2. **Reset stuck active**
   - Verify reset is connected correctly (active-low)
   - Check button/switch state
   - Add ILA to verify reset signal
3. **Incorrect pin assignments**
   - Verify XDC constraints match your board
   - Check VGA connector pinout
   - Ensure all VGA signals are assigned
4. **Monitor incompatibility**
   - Verify monitor supports 640×400 @ 70 Hz
   - Try a different monitor
   - Check VGA cable

11.1.2 Debug Steps

```
// In Vivado TCL console:
create_debug_core u_ila_0 ila
set_property C_DATA_DEPTH 1024 [get_debug_cores u_ila_0]
set_property C_TRIGG_OUT_WIDTH_FROM_CHIP 1 [get_debug_cores u_ila_0]
set_property port_width 1 [get_debug_ports u_ila_0/probe0]
connect_debug_port u_ila_0/probe0 [get_nets pixel_clk]
connect_debug_port u_ila_0/clk [get_nets pixel_clk]
```

Listing 11.1: Insert ILA for Clock Debugging

11.2 Display Shows Noise/Random Pixels

This is actually a good sign! It means:

- VGA timing is working correctly
- Monitor is syncing to the signal
- Framebuffer contains random/uninitialized data

Solution: Initialize the framebuffer from software (see Chapter 6).

11.3 Colors Are Wrong

11.3.1 Symptoms and Fixes

1. **All colors are wrong**
   - Check palette initialization
   - Verify RGB bit assignments in XDC
   - Ensure palette reads are working
2. **Only one color channel works**
   - Check pin assignments for R/G/B signals
   - Verify output signals in simulation
   - Check for stuck-at faults on outputs
3. **Colors change unexpectedly**
   - Check for palette overwrites
   - Verify memory protection if using MMU
   - Look for buffer overruns

11.3.2 Test Palette

```
void test_primary_colors(void) {
    volatile uint32_t *palette = (volatile uint32_t*)PALETTE_BASE;

    // Set test colors
    palette[0] = 0x000;  // Black
    palette[1] = 0xF00;  // Pure red
    palette[2] = 0x0F0;  // Pure green
    palette[3] = 0x00F;  // Pure blue
    palette[4] = 0xFFF;  // White

    // Draw color bars
    uint8_t buffer = 0;
    for (int y = 0; y < 200; y++) {
        for (int x = 0; x < 320; x++) {
            uint8_t color = (x / 64) % 5;
            plot_pixel_320(x, y, color, buffer);
        }
    }
}
```

Listing 11.2: Test Basic Colors

11.4 Screen Tearing in 640×400 Mode

Screen tearing is expected in 640×400 mode since double-buffering is not supported.

11.4.1 Mitigation Strategies

1. **Update during V-blank**
   - Draw only during vertical blanking period
   - Requires fast drawing routines

2. **Partial updates**
   - Only redraw changed areas
   - Minimize update time
3. **Use 320×200 mode**
   - Switch to 320×200 for full double-buffering
   - Trade resolution for smoothness
4. **External memory**
   - Implement triple buffering using SDRAM
   - More complex but eliminates tearing

11.5 Poor Performance

11.5.1 Slow Drawing Speed

1. **Use optimized functions**
   - Use `memset` for fills instead of pixel-by-pixel
   - Use `memcpy` for block copies
   - Enable compiler optimizations (-O2 or -O3)
2. **Reduce overdraw**
   - Don't draw hidden objects
   - Use dirty rectangle tracking
   - Implement z-ordering
3. **Cache frequently used data**
   - Cache sprite data in fast memory
   - Precompute lookup tables
   - Use local variables instead of volatile pointers in loops

11.5.2 AXI Bus Bottleneck

If drawing is still slow:

1. Increase AXI clock frequency (if possible)
2. Use burst writes when supported
3. Enable write-through cache on processor
4. Consider DMA for large transfers

11.6 Build Errors in Vivado

11.6.1 Common Synthesis Errors

1. **Package 'vga_timing_pkg' not found**
   - Ensure compilation order is correct
   - Package must be compiled before entities that use it
   - Check that all .vhdl files are added to project
2. **Block RAM inference failed**
   - Check `ram_style` attributes
   - Verify BRAM code matches Xilinx templates
   - Review synthesis warnings
3. **Timing violations**

- Check clock constraints are applied
- Verify clock periods are correct
- Add pipeline stages if needed
- Use timing analysis to find critical paths

11.7 Getting Help

If problems persist:

1. Review synthesis and implementation reports
2. Check Vivado messages and warnings
3. Use Integrated Logic Analyzer (ILA) to debug
4. Simulate the design in behavioral simulation
5. Post to FPGA forums with detailed information:
    - FPGA board model
    - Vivado version
    - Monitor specifications
    - Error messages
    - Steps to reproduce

Chapter 12

Technical Specifications

This chapter provides detailed technical specifications for the VGA controller, including electrical characteristics (Table 12.1), timing parameters (Table 12.2), and interface specifications (Tables 12.3, 12.4, and 12.5).

12.1 Electrical Specifications

12.1.1 VGA Output Levels

Note: Most VGA monitors expect 0.7V analog signals, but will work with 3.3V digital signals through a simple resistor DAC on the board.

12.1.2 Timing Specifications

12.1.3 Sync Polarity

- H-SYNC: Negative (active low)
- V-SYNC: Positive (active high)

12.2 Memory Specifications

12.2.1 Framebuffer

12.2.2 Palette

12.3 AXI Interface Specifications

12.4 Performance

12.4.1 Throughput

12.4.2 Latency

- AXI write to framebuffer: 1–2 AXI clock cycles
- AXI read from framebuffer: 2–3 AXI clock cycles
- Buffer swap: Instant (next pixel)

Table 12.1: VGA Signal Specifications

Signal	Logic High	Logic Low
H-SYNC	3.3V (LVCMOS33)	0V
V-SYNC	3.3V (LVCMOS33)	0V
R/G/B[3:0]	3.3V (LVCMOS33)	0V

Table 12.2: 640×400 @ 70 Hz Timing

Parameter	**Value**	**Unit**
Horizontal		
Active pixels	640	pixels
Front porch	16	pixels
Sync pulse	96	pixels
Back porch	48	pixels
Total	800	pixels
Line time	31.778	μs
Vertical		
Active lines	400	lines
Front porch	12	lines
Sync pulse	2	lines
Back porch	35	lines
Total	449	lines
Frame time	14.268	ms
Clocks		
Pixel clock	25.175	MHz
Refresh rate	70.09	Hz

Table 12.3: Framebuffer Specifications

Parameter	**Value**
Total size	256 KB
Organization	Byte-addressable
Data width	8 bits per pixel
AXI write modes	Byte, halfword, or word (1-4 bytes)
Access	Dual-port (CPU + VGA)
Technology	FPGA Block RAM
BRAM read latency	1 clock cycle
BRAM write latency	1 clock cycle
AXI transaction	2 cycles (read/single-byte write), 3-5 cycles (multi-byte write)

Table 12.4: Palette Specifications

Parameter	**Value**
Entries	256
Entry width	12 bits (4R:4G:4B)
Storage width	16 bits (word-aligned)
Access	Dual-port (CPU + VGA)
Technology	FPGA Block RAM

Table 12.5: AXI4-Lite Interface Specifications

Parameter	Value
Protocol	AXI4-Lite
Address width	20 bits (1 MB)
Data width	32 bits
Supported operations	Read, Write
Burst support	No (single-beat only)
Byte enables	Yes (for partial writes)
Response	Always OKAY
Clock frequency	50–200 MHz (typical)

Table 12.6: Memory Bandwidth

Operation	Mode	Bandwidth
Display read	320×200 @ 70 Hz	4.48 MB/s
Display read	640×400 @ 70 Hz	17.92 MB/s
CPU write	@ 100 MHz AXI	Up to 400 MB/s

- Mode change: Instant (next pixel)

12.5 Resource Requirements

12.5.1 Minimum FPGA Requirements

- FPGA Family: Xilinx 7-Series or newer
- Block RAM: 64 RAMB36 (or 128 RAMB18)
- LUTs: 1,500 (including 96 LUTRAMs)
- Flip-Flops: 255
- MMCMs/PLLs: 1
- I/O pins: 14 (12 RGB + 2 sync)

12.5.2 Tested Platforms

- Digilent Nexys A7-100T (Artix-7)
- Digilent Arty A7-100T (Artix-7)
- Zynq-7000 devices (tested on Zybo Z7)

Chapter 13

Examples and Tutorials

13.1 Example 1: Hello World

```
// Simple bitmap font (8x8 characters)
const uint8_t font_8x8[128][8] = {
    // ... font data ...
    // 'H' = ASCII 72
    [72] = {0x66, 0x66, 0x66, 0x7E, 0x66, 0x66, 0x66, 0x00},
    // 'E' = ASCII 69
    [69] = {0x7E, 0x60, 0x60, 0x7C, 0x60, 0x60, 0x7E, 0x00},
    // ... etc
};

void draw_char(int x, int y, char c, uint8_t color, uint8_t buffer) {
    for (int row = 0; row < 8; row++) {
        uint8_t bitmap = font_8x8[(int)c][row];
        for (int col = 0; col < 8; col++) {
            if (bitmap & (1 << (7 - col))) {
                plot_pixel_320(x + col, y + row, color, buffer);
            }
        }
    }
}

void draw_string(int x, int y, const char *str,
                 uint8_t color, uint8_t buffer) {
    int pos = 0;
    while (*str) {
        draw_char(x + pos * 8, y, *str, color, buffer);
        str++;
        pos++;
    }
}

int main(void) {
    vga_init();

    // Draw "Hello World!"
    draw_string(100, 96, "Hello World!", 15, 0);

    while(1);
}
```

Listing 13.1: Simple Text Display (8×8 Font)

13.2 Example 2: Bouncing Ball

See Chapter 6, Section 6 for complete bouncing ball example.

13.3 Example 3: Starfield Effect

```
#include <stdlib.h>

typedef struct {
    float x, y;
    float speed;
    uint8_t brightness;
} Star;

#define NUM_STARS 100

Star stars[NUM_STARS];

void init_stars(void) {
    for (int i = 0; i < NUM_STARS; i++) {
        stars[i].x = rand() % 320;
        stars[i].y = rand() % 200;
        stars[i].speed = 0.5 + (rand() % 30) / 10.0;
        stars[i].brightness = 8 + (rand() % 8);
    }
}

void update_stars(void) {
    for (int i = 0; i < NUM_STARS; i++) {
        stars[i].y += stars[i].speed;

        // Wrap around
        if (stars[i].y >= 200) {
            stars[i].y = 0;
            stars[i].x = rand() % 320;
        }
    }
}

void draw_stars(uint8_t buffer) {
    for (int i = 0; i < NUM_STARS; i++) {
        int x = (int)stars[i].x;
        int y = (int)stars[i].y;

        plot_pixel_320(x, y, stars[i].brightness, buffer);
    }
}

void starfield_demo(void) {
    vga_init();

    // Setup grayscale palette
```

```
    for (int i = 0; i < 16; i++) {
        set_palette_entry(i, i, i, i);
    }

    init_stars();

    while (1) {
        uint8_t buffer = db_get_back_buffer();

        // Clear screen
        fast_fill_rect(0, 0, 320, 200, 0, buffer);

        // Update and draw
        update_stars();
        draw_stars(buffer);

        // Swap
        db_swap();
        delay_ms(16);  // ~60 FPS
    }
}
```

Listing 13.2: Scrolling Starfield

13.4 Example 4: Plasma Effect

```
#include <math.h>

void generate_plasma_palette(void) {
    for (int i = 0; i < 256; i++) {
        // Rainbow palette
        float t = i / 256.0;
        uint8_t r = (uint8_t)(7.5 + 7.5 * sin(2 * M_PI * t));
        uint8_t g = (uint8_t)(7.5 + 7.5 * sin(2 * M_PI * t + 2));
        uint8_t b = (uint8_t)(7.5 + 7.5 * sin(2 * M_PI * t + 4));
        set_palette_entry(i, r, g, b);
    }
}

void draw_plasma(float time, uint8_t buffer) {
    for (int y = 0; y < 200; y++) {
        for (int x = 0; x < 320; x++) {
            // Plasma formula
            float value = 0;
            value += sin(x / 16.0);
            value += sin(y / 8.0);
            value += sin((x + y) / 16.0);
            value += sin(sqrt(x*x + y*y) / 8.0);
            value += 4;  // Shift to positive
            value += time;

            // Map to color
```

```
            uint8_t color = (uint8_t)(value * 32) % 256;
            plot_pixel_320(x, y, color, buffer);
        }
    }
}

void plasma_demo(void) {
    vga_init();
    generate_plasma_palette();

    float time = 0;

    while (1) {
        uint8_t buffer = db_get_back_buffer();

        draw_plasma(time, buffer);

        db_swap();

        time += 0.1;
    }
}
```

Listing 13.3: Animated Plasma Effect

13.5 Example 5: Tile Map

```
#define TILE_WIDTH  16
#define TILE_HEIGHT 16
#define MAP_WIDTH   20
#define MAP_HEIGHT  12

// Tile graphics (16x16 pixels each)
const uint8_t tiles[4][256] = {
    // Tile 0: Grass
    { /* 256 bytes of pixel data */ },
    // Tile 1: Stone
    { /* 256 bytes of pixel data */ },
    // Tile 2: Water
    { /* 256 bytes of pixel data */ },
    // Tile 3: Tree
    { /* 256 bytes of pixel data */ },
};

// Map data (which tile at each position)
const uint8_t map[MAP_HEIGHT][MAP_WIDTH] = {
    {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0},
    {0,1,1,1,1,0,0,0,0,0,0,0,0,0,1,1,1,1,0,0},
    {0,1,2,2,1,0,0,3,0,0,0,0,3,0,1,2,2,1,0,0},
    // ... more rows ...
};

```

```
26 void draw_tile(int tile_x, int tile_y, uint8_t tile_id, uint8_t buffer) {
27     int screen_x = tile_x * TILE_WIDTH;
28     int screen_y = tile_y * TILE_HEIGHT;
29 
30     for (int y = 0; y < TILE_HEIGHT; y++) {
31         for (int x = 0; x < TILE_WIDTH; x++) {
32             uint8_t pixel = tiles[tile_id][y * TILE_WIDTH + x];
33             plot_pixel_320(screen_x + x, screen_y + y, pixel, buffer);
34         }
35     }
36 }
37 
38 void draw_map(uint8_t buffer) {
39     for (int ty = 0; ty < MAP_HEIGHT; ty++) {
40         for (int tx = 0; tx < MAP_WIDTH; tx++) {
41             uint8_t tile_id = map[ty][tx];
42             draw_tile(tx, ty, tile_id, buffer);
43         }
44     }
45 }
```

Listing 13.4: Tile-Based Background

Chapter 14

Appendix

14.1 Register Summary

Table 14.1: Complete Register Reference

Register	Address	Description
Framebuffer	0x00000-0x3FFFF	Pixel data (8-bit indexed)
Buffer 0	0x00000-0x0F9FF	First 320×200 buffer
(gap)	0x0FA00-0x1FFFF	Unused (67,072 bytes)
Buffer 1	0x20000-0x2F9FF	Second 320×200 buffer
(gap)	0x2FA00-0x3FFFF	Unused (67,072 bytes)
Palette	0x40000-0x402FF	256 × 16-bit RGB values
Control	0x40200	Bit 0: MODE (0=640×400, 1=320×200)
Display Buffer	0x40204	Bit 0: BUF (buffer select)
IRQ Enable	0x40208	Bit 0: VSIEN (interrupt enable)
IRQ Clear/Status	0x4020C	Bit 0: VSIA (R/W1C, interrupt pending)

14.2 Color Reference

14.2.1 Standard 16-Color Palette Values

Table 14.2: Standard VGA Colors

Index	Name	Hex	R	G	B
0	Black	0x000	0	0	0
1	Blue	0x00A	0	0	10
2	Green	0x0A0	0	10	0
3	Cyan	0x0AA	0	10	10
4	Red	0xA00	10	0	0
5	Magenta	0xA0A	10	0	10
6	Brown	0xA50	10	5	0
7	Light Gray	0xAAA	10	10	10
8	Dark Gray	0x555	5	5	5
9	Bright Blue	0x00F	0	0	15
10	Bright Green	0x0F0	0	15	0
11	Bright Cyan	0x0FF	0	15	15
12	Bright Red	0xF00	15	0	0
13	Bright Magenta	0xF0F	15	0	15

Standard VGA Colors – continued

Index	Name	Hex	R	G	B
14	Yellow	0xFF0	15	15	0
15	White	0xFFF	15	15	15

14.3 Timing Diagrams

14.3.1 Horizontal Timing (640×400 @ 70 Hz)

The horizontal timing waveform is shown in Figure 14.1.

14.3.2 Vertical Timing

The vertical timing waveform is shown in Figure 14.2.

14.4 File Manifest

Table 14.3: VGA Controller File List

File	Purpose
`vga_timing_pkg.vhdl`	Timing constants and type definitions
`vga_timing_gen.vhdl`	VGA sync, blanking, and address generation
`framebuffer_bram.vhdl`	Dual-port framebuffer BRAM (256 KB)
`palette_bram.vhdl`	Dual-port palette RAM (256×12-bit)
`axi_slave.vhdl`	AXI4-Lite slave with burst and RMW support
`register_bank.vhdl`	Control, buffer, and IRQ registers
`interrupt_controller.vhdl`	Edge-detect and W1C interrupt logic
`generic_register.vhdl`	Parameterised flip-flop register primitive
`vga_framebuffer.vhdl`	Framebuffer + palette + register wrapper
`vga_controller.vhdl`	Top-level integration
`vga_controller_tb_comprehensive.vhdl`	Comprehensive simulation testbench
`vga_controller.xdc`	Pin constraints (Nexys A7)
`create_project.tcl`	Vivado project creation script
`README.md`	Quick reference guide

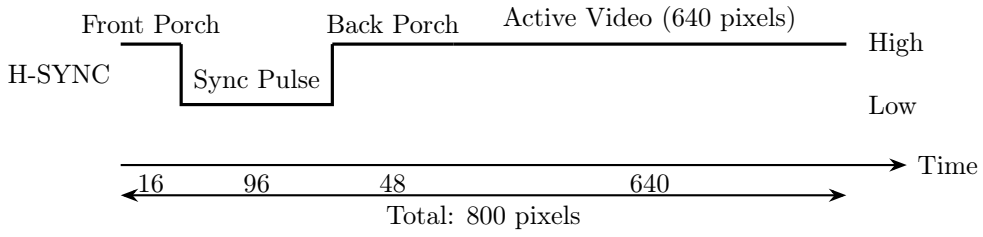


Figure 14.1: Horizontal Timing Diagram (640×400 @ 70 Hz)

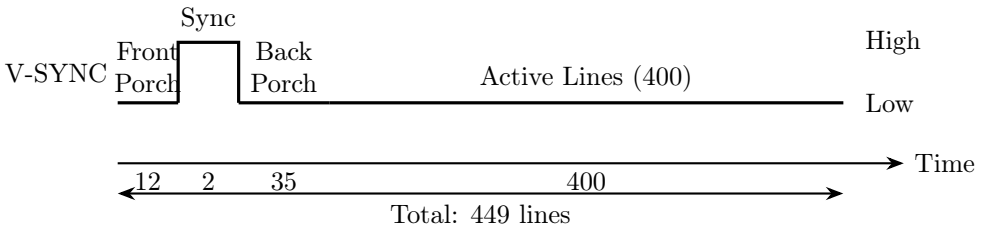


Figure 14.2: Vertical Timing Diagram (640×400 @ 70 Hz)

Table 14.4: Document Revision History

Version	Date	Changes
1.0	2026-02-13	Initial release
1.1	2026-02-20	Updated base address to `0x48000000`; split Interrupt Control into separate IRQ Enable (`0x40208`) and IRQ Clear/Status (`0x4020C`) registers; corrected Buffer 1 start address to `0x20000` (128 KB boundary); corrected palette address range to `0x40000-0x401FF`; updated file manifest

14.5 Glossary

AXI Advanced eXtensible Interface - ARM's standard for on-chip communication
BRAM Block RAM - Dedicated memory blocks in FPGAs
Double-Buffering Technique using two framebuffers to eliminate tearing
Frame Buffer Memory holding pixel data for display
H-SYNC Horizontal Synchronization pulse
Indexed Color Color system using palette lookups
Palette Lookup table mapping pixel indices to RGB colors
Pixel Clock Clock rate at which pixels are output (25.175 MHz)
Tearing Visual artifact from updating visible framebuffer
V-SYNC Vertical Synchronization pulse
VGA Video Graphics Array - Analog display standard
VHDL VHSIC Hardware Description Language

14.6 References

1. VESA Monitor Timing Specification
2. Xilinx 7-Series FPGA Documentation
3. AXI4-Lite Protocol Specification (ARM IHI 0022E)
4. VGA Signal Specification
5. Digilent Nexys A7 Reference Manual

14.7 Revision History